

AN2DL - First Challenge Report

thegradientdescenters

Alberto Occhipinti, Giulia Putelli, Matteo Morini, Tommaso Rossetti

albertoochipinti, giuliaputelli, ilraccoglitore, tommasorossetti

301511, 302959, 308227, 301319

November 17, 2025

1 Introduction

This project addresses the first challenge of the Artificial Neural Networks and Deep Learning course: the *Pirate Pain Prediction* task. The goal is to classify the level of pain experienced by a subject—categorized as `no_pain`, `low_pain`, or `high_pain`—based on time-series data.

The dataset provides two types of features:

- **Time-series features:** 31 joint sensor readings and 4 pain survey responses, recorded over time.
- **Static features:** 3 categorical attributes for each subject (number of legs, hands, and eyes).

Our objective was to build a deep learning model that accurately classifies pain levels, maximizing the **weighted F1-score**. Our final approach utilizes a Gated Recurrent Unit (GRU) network enhanced with an attention mechanism and a dedicated embedding path for static features, all validated using a robust cross-validation strategy.

2 Problem Analysis

Initial analysis of the dataset revealed three primary challenges:

1. **Subject-Dependent Data:** The data is grouped by `sample_index`, representing individual subjects. Measurements from the same subject are highly correlated. A simple random split for validation would lead to data leakage and overly optimistic results.
2. **Class Imbalance:** The dataset is imbalanced, with a significantly larger number of samples for the `no_pain` class compared to `low_pain` and `high_pain`.

To address these, we implemented a specific preprocessing pipeline.

2.1 Data Preprocessing

- **Categorical Mapping:** The static string features (`n_legs`, `n_hands`, `n_eyes`) were mapped to integers (e.g., "two" → 2, "one+peg_leg" → 1).
- **Normalization:** All 35 time-series features (joints and surveys) were scaled using Min-Max normalization. The scaler was fit only on the training data within each cross-validation fold to prevent leakage.
- **Windowing:** We used a **sliding window** approach. We defined a `WINDOW_SIZE` of 44 timesteps and a `STRIDE` of 22, creating overlapping sequences. Sequences were zero-padded to ensure all data could be processed into full windows.

3 Method

Our model, a `RecurrentClassifierWithAttention`, is designed to process both temporal and static data in parallel before fusing them for a final classification.

3.1 Model Architecture

The architecture consists of three main parts:

1. **Temporal Path:** A multi-layer Gated Recurrent Unit (GRU) processes the windowed time-series data (35 features). We used a 2-layer GRU with a hidden size of 128 and a high dropout rate of 0.5 for regularization and generalization.
2. **Static Path:** The 3 integer-mapped categorical features are passed through parallel `nn.Embedding`

layers. The resulting embeddings are concatenated and projected by a small MLP to create a learned static feature representation.

3. **Attention and Fusion:** We use an `AttentionEncoderOnly` layer. The GRU’s final hidden state is used as a query to attend over all its encoder outputs (hidden states from all timesteps). This produces a weighted context vector, summarizing the most relevant parts of the sequence. This context vector is then **concatenated** with the projected static features.

Finally, a linear classifier layer takes this fused vector and outputs the logits for the 3 pain classes.

3.2 Training Strategy

Our training process incorporated several key techniques:

- **Loss Function:** We used `nn.CrossEntropyLoss` with two modifications: **class weights** were applied to counteract the class imbalance, and **label smoothing** (0.1) was used to regularize the model and prevent overconfidence.
- **Optimizer:** We chose `AdamW`, which integrates L2 regularization (weight decay) directly into the optimizer. We set $\lambda_{L2} = 10^{-4}$.
- **Regularization:** Our main tool against overfitting was a high **Dropout Rate of 0.5** in the GRU.
- **Stability:** We used **gradient clipping** (max norm = 1.0) to prevent exploding gradients and `torch.amp` (automatic mixed precision) for faster training.
- **Validation:** To solve the subject-dependent data challenge, we used a **5-fold Stratified Group K-Fold** cross-validation. The data was grouped by `sample_index` (subject) and stratified by the `label`. This ensures that all data from a single subject appears in either the training or validation set, but never both, providing a realistic estimate of generalization.

4 Experiments

We iterated through several model designs. Our baseline was a simple GRU, which we improved by adding the attention mechanism. We then experimented with several strategies:

- **Failed Attempts:** We tested adding noise to the training data and using an automatic learning rate scheduler, but neither improved the F1-score. Noticing the lack of samples with one missing joint we tried using SMOTE and ADASYN, but they didn’t work well. A full hyperparameter sweep was computationally infeasible. We also implemented a CNN-GRU hybrid model as suggested, but it did not outperform the GRU-Attention model.

- **Successful Strategy:** The most significant improvement came from tackling overfitting. By increasing the **dropout rate to 0.5**, we substantially improved the validation F1-score and model generalization.

Our final model was trained using the 5-fold cross-validation with the hyperparameters listed in Table 1. Early stopping with a patience of 150 epochs was used to select the best model from each fold based on its validation F1-score.

Table 1: Final Model Hyperparameters.

Hyperparameter	Value
RNN Type	GRU
Bidirectional	False
Learning Rate	1e-3
Epochs (Max)	1500
Patience	150
Hidden Layers	2
Hidden Size	128
Dropout Rate	0.5
L2 Lambda	1e-4
Window Size	44
Stride	22
Batch Size	64

5 Results

We initially experimented with a standard training-validation-test split to observe the model’s behavior on a local test case. For the final submission, we shifted to using only training and validation sets, employing a 5-fold cross-validation (CV) strategy. This 5-fold cross-validation provided a robust measure of our model’s performance. The validation F1-scores for each fold are shown in Table 2. Our model achieved a mean weighted F1-score of **0.9381 ± 0.0140**.

Table 2: 5-Fold Cross-Validation Results (Weighted F1-Score).

Fold	Best Validation F1-Score
Split 1	0.9631
Split 2	0.9426
Split 3	0.9243
Split 4	0.9326
Split 5	0.9276

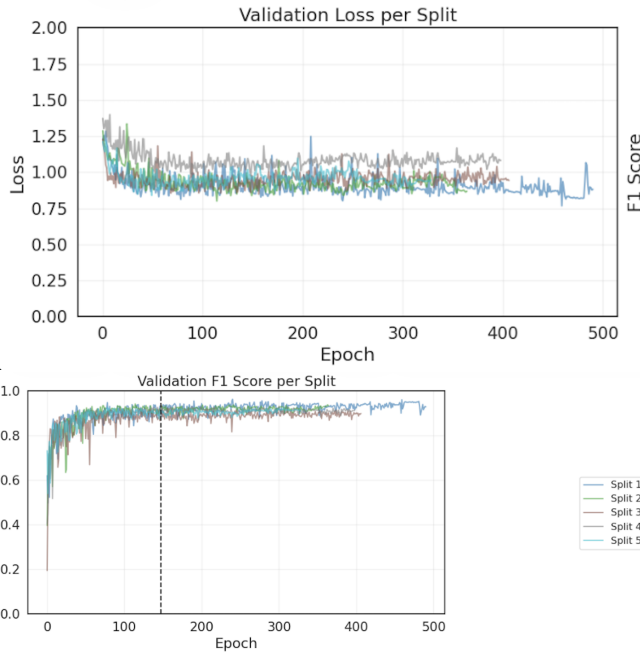


Figure 1: Validation F1-score (right) and Loss (left) per epoch for each of the 5 cross-validation folds.

5.1 Final Submission

For the final submission, we did not rely on a single model. Instead, we created an **ensemble** of the 5 best models saved from each fold. Our prediction pipeline was as follows:

1. **Soft Voting(Ensemble):** For each individual test sequence, we averaged the softmax probabilities from all 5 models.
2. **TTA Aggregation(by subject):** Since each `sample_index` in the test set corresponds to multiple (overlapping) sequences, we took the **majority vote (mode)** of all sequence predictions belonging to that subject.

This ensemble approach leverages the diverse perspectives of models trained on different data folds, leading to a more robust and generalized final prediction.

6 Discussion

We successfully developed a model for classifying pirate pain levels from joints data. The final model, a GRU with an attention mechanism and static feature embedding, proved highly effective.

The key factors contributing to our model’s success were:

- Using **Stratified Group K-Fold** cross-validation to get a reliable performance estimate and prevent subject-based data leakage.
- Handling class imbalance with a **weighted loss function**.
- Controlling overfitting through **high dropout (0.5)** and **L2 regularization**.
- Creating a **soft-voting ensemble** from the 5-fold models for the final submission, which is more robust than any single model.

7 Conclusions

We built and refined a GRU-based model with attention to classify pain levels from multivariate time-series, supported by careful data cleaning and group-aware validation. Our work shows that regularization, hyperparameter tuning, and simple ensembling significantly improve generalization. Future developments could include transformer architectures, better handling of label imbalance, and self-supervised pretraining.